Statement of Python Code

Student ID: 39991938

January 20, 2026

customKNN_model.py

Our custom KNN model is based on main KNN classifier principles (Zhang et al. (2016); IBM (2024); Halder et al. (2024); Uddin et al. (2022); Cunningham and Delany (2021); Abu Alfeilat et al. (2019); Lopes and Ribeiro (2023)) and has the following functions: __init__ for setup, fit() for storing training data, euclideanDistance() and manhattanDistance() for calculating distances, distanceType() for picking which distance to use, predict() for making predictions and implementing voting types, and get_params() for returning model settings.

The `__init__` function stores the model settings. The pattern `self.k = k` was copied from tutorials (Akinkugbe (2024); Towards Data Science (2025)). Having three parameters (k, metric, weights) was adapted from scikit-learn (Scikit-learn Developers (2024)). The error check for negative k and setting `X_train` and `Y_train` to None at the start are my own work.

The fit() function is mostly a direct copy from Akinkugbe (2024); Towards Data Science (2025). I added an error check that makes sure the number of data points matches the number of labels.

The understanding of Euclidean and Manhattan distance came from academic papers (Singh et al. (2017); Hu et al. (2016); Chomboon et al. (2015)). The code for both distance functions was copied from online sources (EITCA Academy (2023); CodePointTech (2025)).

The predict() function follows standard KNN logic adapted from online tutorials (Akinkugbe (2024); Towards Data Science (2025); Ahmed (2024)). It calculates distances to all training points, sorts them, and picks the k nearest neighbours. For uniform voting, it uses `Counter` to count labels and pick the most common one. For distance-weighted voting, it uses the 1/d idea from Dudani (1976), where closer neighbours count more. The small epsilon value (0.0001) to avoid dividing by zero was adapted from Ahmed (2024). My own additions are the error checks that make sure fit() was called before predict() and that k is not bigger than the training set.

The get_params() function returns the model settings as a dictionary. This idea was adapted from scikit-learn (Scikit-learn Developers (2024)).

In summary, I copied the distance formulas and basic KNN patterns from online sources. What I built myself is the error handling, putting both distance metrics and voting types into one class, and deciding how to set up the experiments.

experiments.py

This file runs all experiments comparing our custom KNN against sklearn's KNN under identical conditions. It imports our custom model from customKNN_Model.py and dataset functions from dataset_loader.py. Configuration variables (k_Range, distanceMetrics, votingTypes, trainSplits, randomSeed) define what hyperparameter combinations to test.

The file has these functions: splittingDatasets() splits data into train and test sets - this is a direct copy from sklearn docs (scikit-learn developers (2024g)). scalingFeatures() standardizes features using StandardScaler - also a direct copy (scikit-learn developers (2024f)). metricsCalculations() calculates accuracy, precision, recall and F1 - direct copy from sklearn (scikit-learn developers (2024)). memoryUsageAmount() measures memory using sys.getsizeof() - direct copy from Python docs (Python Software Foundation (2024e)).

runModel() - it runs all experiments for one dataset. It uses four nested loops to test every combination of training size, distance metric, voting type, and k value. This nested loop pattern is standard practice (scikit-learn developers (2024h)). For each combination, it creates the model (our custom KNN or sklearn's KNeighborsClassifier (Scikit-learn Developers (2024))), trains it, predicts, and measures timing with time.perf_counter() (Python Software Foundation (2024f)). Results go into a list with 20 columns. There's also a safety check that skips if k is bigger than the training set. The specific values we test and the 20-column format are our own choices.

runningModels_and_Datasets() loops through all five datasets and calls runModel() twice for each - once for custom, once for sklearn. Results get stored in two separate lists. This function is original work.

csvProduction() saves results to CSV files. It uses csv.writer with writerow() for the header then loops through the data - this pattern is a direct copy from Python docs (Python Software Foundation (2024a)). The 20 column names we chose are original.

main() just calls runningModels_and_Datasets() then csvProduction() twice to save both result files. The if __name__ == "__main__" pattern at the bottom is standard Python (Python Software Foundation (2024c)) - it makes sure main() only runs when we execute the file directly, not when we import from it.

dataset_loader.py

This file loads all five datasets used in our experiments. It imports numpy, os, pandas, and four sklearn dataset loaders. It has two functions: load_dataset() which loads any dataset by name, and get_available_datasets() which returns a list of all dataset names.

The load_dataset() function uses if/elif statements to check which dataset was requested. For Iris, Wine, Digits, and Breast Cancer, we use sklearn's built-in loaders. The pattern data = load_iris() then X = data.data, Y = data.target, feature_names = data.feature_names, and target_names = data.target_names is a direct copy from sklearn documentation (scikit-learn developers (2024c), scikit-learn developers (2024d), scikit-learn developers (2024b), scikit-learn developers (2024a)). For Digits, the dataset has no meaningful feature names so we create generic ones like "pixel_0", "pixel_1" using an f-string list comprehension - the f-string syntax is standard Python (Python Software Foundation (2024b)).

For the Nursery dataset, we load it from a CSV file. We check multiple possible file paths using os.path.exists() which is standard Python (Python Software Foundation (2024d)). We load the file with pd.read_csv(path, header=None) - direct copy from pandas docs (pandas development team (2024b)). We split features and target using iloc[:, :-1] and iloc[:, -1] - direct copy from pandas docs (pandas development team (2024a)). The Nursery data has text values, so we convert them to numbers using LabelEncoder with fit_transform() in a loop through each column - direct copy from sklearn docs (scikit-learn developers (2024e)). We create an empty array first with np.zeros_like() - direct copy from numpy docs (NumPy Developers (2024g)).

For each dataset we create an info dictionary storing name, n_samples (X.shape[0]), n_features (X.shape[1]), n_classes (len(np.unique(Y))), and size_category. The shape and unique calls are standard numpy (NumPy Developers (2024d), NumPy Developers (2024f)).

The get_available_datasets() function returns a list of the five dataset names as strings. This is original work.

What is original: the if/elif structure to load different datasets by name, the info dictionary design, the list of possible file paths for Nursery, the choice of five datasets (two small, two medium, one large), and the size_category labels.

test_knn.py

This file contains unit tests for our custom KNN model using the pytest framework (pytest developers (2024b)). It imports pytest and our custom KNN model, then has 6 test functions to check if the main parts of our code work correctly.

The first two tests check if our distance calculations are correct. For euclidean distance, we use a 3-4-5 right triangle - the distance from (0,0) to (3,4) should equal exactly 5. For manhattan distance, we check that going from (0,0) to (3,4) equals 7.

Two tests check basic fit and predict functions. test_fit_stores_data() checks that after calling fit(), the training data is stored in the model. test_predict_returns_correct_length() checks that if we give predict() two test points, we get two predictions back.

The last two tests check voting. test_uniform_voting() checks that majority vote picks the most common class among neighbours. test_distance_weighted_voting() checks that closer neighbours get more power when using distance weighting.

What is original: we chose which 6 things to test, we picked the specific test values, and we wrote it. The testing patterns like assert are standard pytest practice (pytest developers (2024a)).

decisionBoundaries_data.py

This file creates data for visualizing decision boundaries in R. Decision boundaries show how a classifier divides the feature space into different classes (Dudani (1976), Gou et al. (2011)).

The code uses standard numpy functions according to their documentation: np.linspace() for creating evenly spaced points (NumPy Developers (2024b)), np.meshgrid() for creating 2D grids (NumPy Developers (2024c)), np.ravel() for flattening arrays (NumPy Developers (2024e)), and np.c_ for combining coordinates (NumPy Developers (2024a)). Data loading uses standard sklearn patterns (scikit-learn developers (2024c)).

The parts we made ourselves include: how we save the results to CSV for R to use, the idea of storing custom and sklearn predictions together, which k values we picked, and how we split the code into separate functions.

References

Abu Alfeilat, H. A., Hassanat, A. B. A., Lasassmeh, O., Tarawneh, A. S., Alhasanat, M. B., Eyal Salman, H. S., and Prasath, V. B. S. (2019). Evaluation of k-nearest neighbour classifier performance for heterogeneous data sets. *SN Applied Sciences*, 1(12):1559.

Ahmed, S. (2024). Demystifying k neighbors classifier (knn): Theory and python implementation from scratch. https://medium.com/@sahin.samia/demystifying-k-neighbors-classifier-knn-theory-and-python-implementation-from-scratch-f5e76d6f2d48.

Akinkugbe, A. (2024). Build k-nearest neighbors knn from scratch. https://python.plainenglish.io/build-k-nearest-neighbors-knn-from-scratch-10dbc5b21254. Python in Plain English.

Chomboon, K., Chujai, P., Teerarassammee, P., Kerdprasop, K., and Kerdprasop, N. (2015). An empirical study of distance metrics for k-nearest neighbor algorithm. In *Proceedings of the 3rd International Conference on Industrial Application Engineering*, pages 280–285.

CodePointTech (2025). Mastering manhattan distance in python: A practical guide. https://codepointtech.com/mastering-manhattan-distance-in-python-a-practical-guide/.

Cunningham, P. and Delany, S. J. (2021). k-nearest neighbour classifiers: A tutorial. *ACM Computing Surveys*, 54(6):1–25.

Dudani, S. A. (1976). The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-6(4):325–327.

EITCA Academy (2023). How do we calculate the euclidean distance between two data points using basic python operations? https://eitca.org/artificial-intelligence/eitc-ai-mlp-machine-learning-with-python/programming-machine-learning/programming-own-k-nearest-neighbors-algorithm/examination-review-programming-own-k-nearest-neighbors-algorithm/how-do-we-calculate-the-euclidean-distance-between-two-data-points-using-basic-python-operations/.

Gou, J., Xiong, T., and Kuang, Y. (2011). A novel weighted voting for k-nearest neighbor rule. *Journal of Computers*, 6(5):833–840.

Halder, R. K., Uddin, M. N., Uddin, M. A., Aryal, S., and Khraisat, A. (2024). Enhancing k-nearest neighbor algorithm: a comprehensive review and performance analysis of modifications. *Journal of Big Data*, 11:113.

Hu, L.-Y., Huang, M.-W., Ke, S.-W., and Tsai, C.-F. (2016). The distance function effect on k-nearest neighbor classification for medical datasets. *SpringerPlus*, 5(1):1304.

IBM (2024). What is the k-nearest neighbors algorithm? https://www.ibm.com/think/topics/knn.

Lopes, N. and Ribeiro, B. (2023). kNN classification: a review. *Annals of Mathematics and Artificial Intelligence*, 91:791–829.

NumPy Developers (2024a). numpy.c_. https://numpy.org/doc/stable/reference/generated/numpy.c_.html.

NumPy Developers (2024b). numpy.linspace. https://numpy.org/doc/stable/reference/generated/numpy.linspace.html.

NumPy Developers (2024c). numpy.meshgrid. https://numpy.org/doc/stable/reference/generated/numpy.meshgrid.html.

NumPy Developers (2024d). numpy.ndarray.shape. https://numpy.org/doc/stable/reference/generated/numpy.ndarray.shape.html.

NumPy Developers (2024e). numpy.ravel. https://numpy.org/doc/stable/reference/generated/numpy.ravel.html.

NumPy Developers (2024f). numpy.unique. https://numpy.org/doc/stable/reference/generated/numpy.unique.html.

NumPy Developers (2024g). numpy.zeros_like. https://numpy.org/doc/stable/reference/generated/numpy.zeros_like.html.

pandas development team (2024a). pandas.dataframe.iloc. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.iloc.html.

pandas development team (2024b). pandas.read_csv. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html.

pytest developers (2024a). How to write and report assertions in tests. https://docs.pytest.org/en/stable/how-to/assert.html.

pytest developers (2024b). pytest: helps you write better programs. https://docs.pytest.org/.

Python Software Foundation (2024a). csv — csv file reading and writing. https://docs.python.org/3/library/csv.html.

Python Software Foundation (2024b). Formatted string literals. https://docs.python.org/3/tutorial/inputoutput.html.

Python Software Foundation (2024c). __main__ — top-level code environment. https://docs.python.org/3/library/__main__.html.

Python Software Foundation (2024d). os.path — common pathname manipulations. https://docs.python.org/3/library/os.path.html.

Python Software Foundation (2024e). sys — system-specific parameters and functions. https://docs.python.org/3/library/sys.html.

Python Software Foundation (2024f). time — time access and conversions. https://docs.python.org/3/library/time.html.

scikit-learn developers (2024). Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html.

Scikit-learn Developers (2024). Nearest neighbors — scikit-learn documentation. https://scikit-learn.org/stable/modules/neighbors.html.

scikit-learn developers (2024a). sklearn.datasets.load_breast_cancer. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html.

scikit-learn developers (2024b). sklearn.datasets.load_digits. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html.

scikit-learn developers (2024c). sklearn.datasets.load_iris. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html.

scikit-learn developers (2024d). sklearn.datasets.load_wine. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html.

scikit-learn developers (2024e). sklearn.preprocessing.labelencoder. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html.

scikit-learn developers (2024f). Standardscaler. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.

scikit-learn developers (2024g). train_test_split. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html.

scikit-learn developers (2024h). Tuning the hyper-parameters of an estimator. https://scikit-learn.org/stable/modules/grid_search.html.

Singh, A., Halgamuge, M. N., and Lakshmiganthan, R. (2017). Effects of distance measure choice on KNN classifier performance - a review. *arXiv preprint arXiv:1708.04321*.

Towards Data Science (2025). Create your own k-nearest neighbors algorithm in python. https://towardsdatascience.com/create-your-own-k-nearest-neighbors-algorithm-in-python-eb7093fc6339/.

Uddin, S., Haque, I., Lu, H., Moni, M. A., and Gide, E. (2022). Comparative performance analysis of k-nearest neighbour (knn) algorithm and its different variants for disease prediction. *Scientific Reports*, 12:6256.

Zhang, S., Li, X., Zong, M., Zhu, X., and Wang, R. (2016). Introduction to machine learning: K-nearest neighbors. *Annals of Data Science*, 3(2):179–206.